ARTIFICIAL INTELLIGENCE

Complete Final Exam Study Guide

State Space Search · Heuristic Search · Adversarial Search

Local & Evolutionary Search · Machine Learning · ANN & Backpropagation

CNN · Clustering · Performance Evaluation

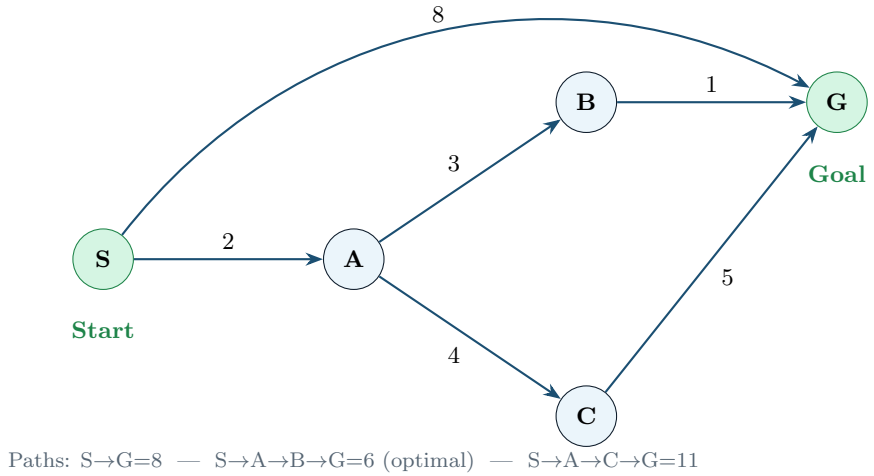
Extreme Detail · TikZ Diagrams & PGFPlots · Dry Runs · Easy to Hard Exam Questions

Backpropagation: Sigmoid Only · No Python, Pure LaTeX

0. 1

1. 1

A **state space** is a 5-tuple: $\langle S, s_0, A, T, G \rangle$ where S=all states, s_0=initial state, A=actions/operators, T=transition function, G=goal states. A **path** is a sequence of states; a **solution** is any path from s_0 to $g \in G$; an **optimal solution** has minimum cost.



Paths: S→G=8 — S→A→B→G=6 (optimal) — S→A→C→G=11

Nodes=states, edges=operators with costs. Optimal path is S→A→B→G with total cost 6, not the direct edge costing 8.

Four Key Properties of Any Search Algorithm

Property	Definition
Completeness	Guarantees finding a solution *if one exists*
Optimality	Guarantees the solution has *minimum cost*
Time Complexity	Nodes expanded as function of b (branching factor), d (shallowest goal depth), m (max depth)
Space Complexity	Maximum nodes held in memory simultaneously

Key Terminology

Frontier (Open List): Nodes discovered but not yet expanded.
Explored (Closed List): Nodes already expanded (graph search only).
Branching factor b: Average number of successors per node.
Depth d: Number of edges from root to shallowest goal.
Max depth m: Maximum depth of any path in the search tree.

2. 1

Blind search has **no information** about goal location. It only knows whether a state is a goal or not.

2.1 1

DFS explores as **deep as possible** before backtracking. Uses a **stack (LIFO)**.

Key Insight: When you push successors and immediately pop, you always go deeper. The stack only holds the current path plus unexplored siblings at each level — hence linear space $O(bm)$.

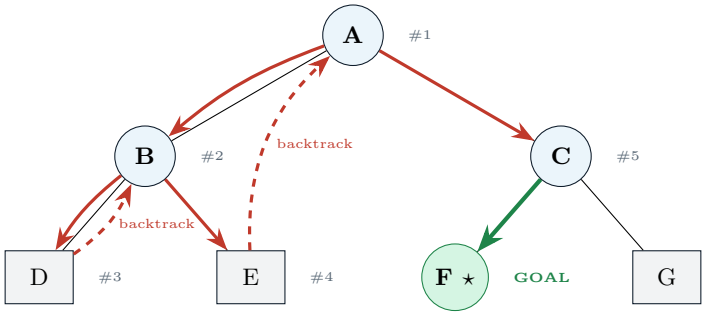
DFS Algorithm

1. Push initial state onto stack
2. If stack empty ⇒ return failure
3. Pop top node n
4. If n is goal ⇒ return solution
5. Mark n visited; push unvisited successors
6. Go to step 2

DFS Properties

Property	Value
Complete?	No (infinite loops)
Optimal?	No
Time	$O(b^m)$
Space	$\boldsymbol{O(bm)}$ – best!

2.1.1 1DFS Full Tree Diagram with Traversal Order



Complete DFS Trace: Initial

Step	Stack (top first)	Visited	Action
0	[A]	{}	Initialize: push A
1	[C, B]	{A}	Pop A; not goal; push successors C,B (B on top)
2	[E, D, C]	{A,B}	Pop B; not goal; push D,E
3	[D, C]	{A,B,E}	Pop E; leaf, not goal
4	[C]	{A,B,E,D}	Pop D; leaf, not goal
5	[G, F]	{A,B,E,D,C}	Pop C; not goal; push F,G
6	[G]	{A,B,E,D,C,F}	Pop F = **GOAL! Return path: A→C→F**

Note on push order: We push right child first so left child is on top and explored first. Here we push B before C so B is popped first — deeper left branch explored before right.

Q: [Easy] What data structure does DFS use? Why does it explore depth-first?
A: A Stack (LIFO). Successors are immediately on top, so DFS dives deeper before trying siblings.

Q: [Medium] DFS tree: $b = 3$, max depth $m = 4$. How many nodes in memory at once?
A: $O(bm) = 3 \times 4 = 12$ nodes. At depth k, stack holds: current path ($k+1$ nodes) + at most $b-1$ siblings per level $= k + 1 + (b-1)k = bk + 1$. For $b = 3$, $m = 4$: $3 \times 4 + 1 = 13$ maximum. DFS's **linear space** is its greatest strength.

Q: [Hard] Graph: 1→2,3 — 2→4,5 — 3→6,7 (unit costs). Goal=6. Trace DFS left-first. Is path optimal? Why not guaranteed?
A: Stack=[1]. Pop 1→push [3,2]. Pop 2→push [5,4,3]. Pop 4=leaf. Pop 5=leaf. Pop 3→push [7,6]. Pop 6=GOAL. Path: 1→3→6, depth=2, cost=2. Happens to be optimal here. But DFS gives NO general optimality guarantee — it could find 1→3→7 first if push order were different, or a deeper path first in a weighted graph.

2.2 1

BFS explores all nodes at depth d before depth $d+1$. Uses a **queue (FIFO)**.

Key Insight: Since BFS explores level by level, the first time it reaches the goal it must have taken the fewest edges — hence optimal for uniform costs.

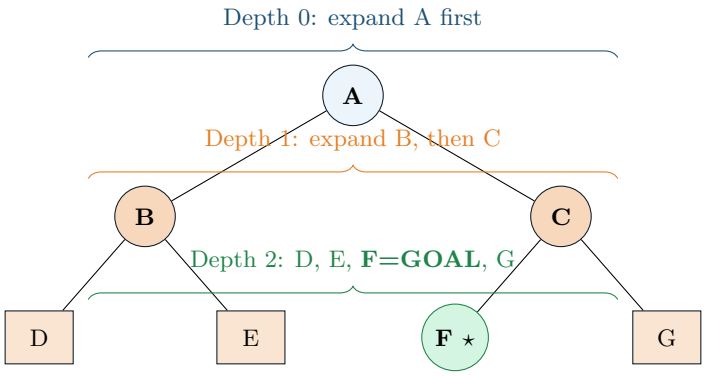
BFS Algorithm

1. Enqueue initial state
2. If queue empty ⇒ return failure
3. Dequeue front node n
4. If n is goal ⇒ return solution
5. Enqueue all unvisited successors (L→R)
6. Go to step 2

BFS Properties

Property	Value
Complete?	**Yes**
Optimal?	**Yes** (equal costs)
Time	$O(b^d)$
Space	$O(b^d)$ – worst!

2.2.1 1BFS Level-by-Level Diagram

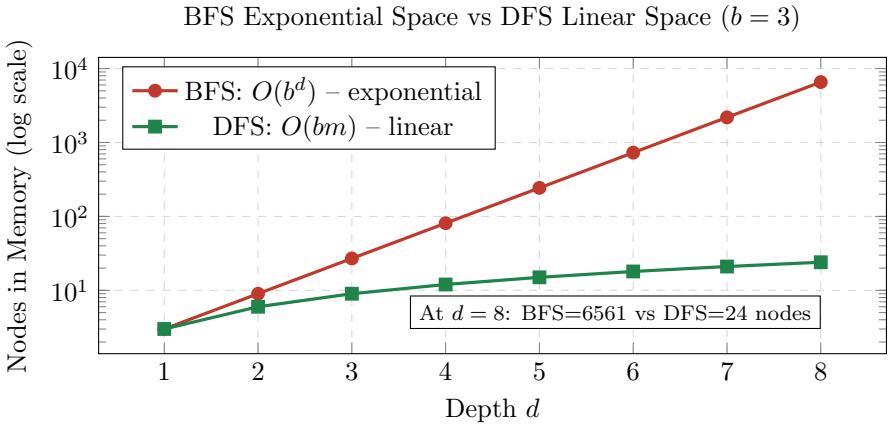


BFS Trace: same graph

Step	Queue (front first)	Visited	Action
0	[A]	{}	Enqueue A
1	[B, C]	{A}	Dequeue A → enqueue B,C
2	[C, D, E]	{A,B}	Dequeue B → enqueue D,E
3	[D, E, F, G]	{A,B,C}	Dequeue C → enqueue F,G
4	[E, F, G]	{A,B,C,D}	Dequeue D → leaf
5	[F, G]	{A,B,C,D,E}	Dequeue E → leaf
6	[G]	{...,F}	Dequeue F = **GOAL!** **Path: A→C→F**

*BFS dequeued 6 nodes; DFS found the goal in fewer steps (by luck of path order). But BFS **guarantees the shallowest** (fewest-edge) path. DFS offers no such guarantee.*

2.2.2 1BFS vs DFS Memory Usage – Graph



Q: [Easy] Why is BFS optimal for equal-cost but NOT for weighted graphs?
A: BFS minimizes the *number of edges* (steps). In weighted graphs, fewer edges ≠ lower total cost. Example: A→B cost=100, A→C→B cost=1+1=2. BFS finds A→B first (1 edge) but it is not optimal.

Q: [Medium] $b = 4$, goal at depth $d = 5$. Worst-case BFS nodes in memory?

A: $O(b^d) = 4^5 = 1024$ nodes. All nodes at depth d must be in the queue simultaneously, plus all at $d-1$ that haven't been expanded yet.

Q: [Hard] Graph: 1→2,3,4 — 2→5,6 — 3→7 — 4→8. Goal=7. Compare BFS vs DFS (left-first). Which expands fewer nodes? Which guarantees optimality?
A: **BFS**: enqueue[2,3,4], dequeue 2→enqueue[3,4,5,6], dequeue 3→enqueue[4,5,6,7], dequeue 4, dequeue 5, dequeue 6, dequeue 7=GOAL. 7 dequeues. **DFS** (left-first push: right child first): Stack=[1]. Pop 1→push[4,3,2]. Pop 2→push[6,5,4,3]. Pop 5=leaf. Pop 6=leaf. Pop 4→push[8,4,3]. Pop 8=leaf. Pop 4=visited skip. Pop 3→push[7,4]. Pop 7=GOAL. DFS expands fewer nodes here. But BFS guarantees shortest path 1→3→7 (depth 2). DFS has no such guarantee.

2.3 1

IDS runs DFS repeatedly with increasing depth limits $L = 0, 1, 2, \ldots$ until goal found. Gets the **best of both**: DFS space + BFS completeness and optimality.

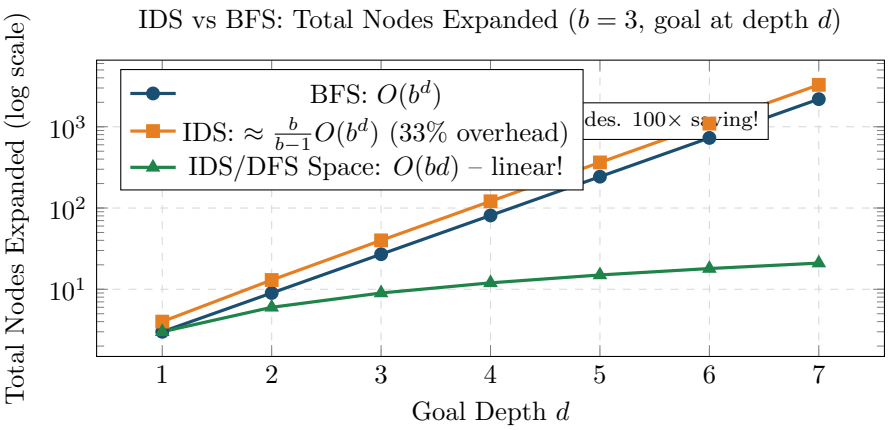
2.3.1 1IDS Iteration Visualization

L=0: Expand only root A. Not found. (1 node)

L=1: Expand A; push B,C. Pop B (dead end depth 1); pop C (dead end). Not found. (3 nodes)

L=2: A→B→D(dead), B→E(dead); C→**F=GOAL!** Path: A→C→F (6 nodes explored)

2.3.2 1IDS vs BFS: Re-expansion Cost and Space Saving



IDS expands $\frac{b}{b-1}$ times more nodes than BFS. For $b = 3$: 33% overhead, $b = 10$: 11% overhead. Re-expansion penalty is tiny compared to the massive space saving.

IDS Node Count Formula: b

Each node at depth k is re-expanded $(d - k + 1)$ times across all iterations.
Total $= (d+1) \cdot b^0 + d \cdot b^1 + (d-1) \cdot b^2 + \ldots + 1 \cdot b^d$
$= 5(1) + 4(3) + 3(9) + 2(27) + 1(81) = 5 + 12 + 27 + 54 + 81 =$ **179**
BFS would be $1 + 3 + 9 + 27 + 81 = 121$. IDS uses 48% more expansions but uses only $O(bd) = 12$ memory vs $O(b^d) = 81$.

Q: [Easy] What two algorithms does IDS combine? What does it inherit from each?
A: DFS (linear space $O(bm)$) + BFS (completeness + optimality for uniform costs).

Q: [Medium] IDS $b = 3$, $d = 4$. Total nodes generated? Compare to BFS.
A: IDS: 179. BFS: 121. IDS generates 48% more nodes but uses only $O(bd) = 12$ memory vs BFS's $O(b^d) = 81$.

Q: [Hard] When does IDS fail to find the optimal solution?
A: Non-uniform step costs. Example: A→B cost=1, A→C cost=1, B→G cost=100, C→G cost=1. Both

goals at depth 2. IDS finds B→G (cost 101) before seeing C→G (cost 2) if B is explored first. **Use UCS for weighted graphs**.

2.4 1

UCS always expands the node with **lowest cumulative path cost** $g(n)$. Uses a **priority queue (min-heap)** ordered by $g(n)$. Generalizes BFS to weighted graphs.

Critical rule: Only declare goal when **dequeued**, not when inserted into the priority queue.

UCS Algorithm

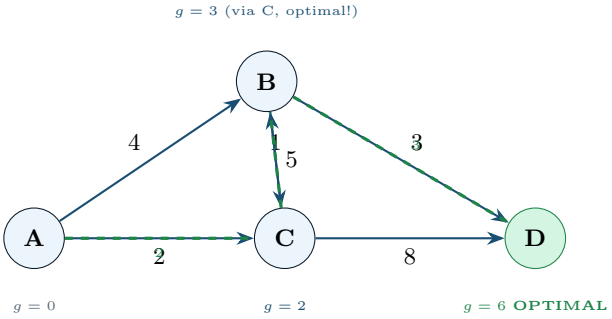
1. Insert start with $g(s_0) = 0$ into PQ
2. If PQ empty ⇒ failure
3. Extract node n with min g
4. If n is goal ⇒ return $g(n)$
5. For each successor s: $g(s) = g(n) + cost(n, s)$; insert/update in PQ
6. Go to step 2

UCS Properties

Property	Value
Complete?	**Yes** (costs>0)
Optimal?	**Yes**
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
Space	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

C^*=optimal cost, ε=min step cost.

2.4.1 1UCS Weighted Graph Dry Run

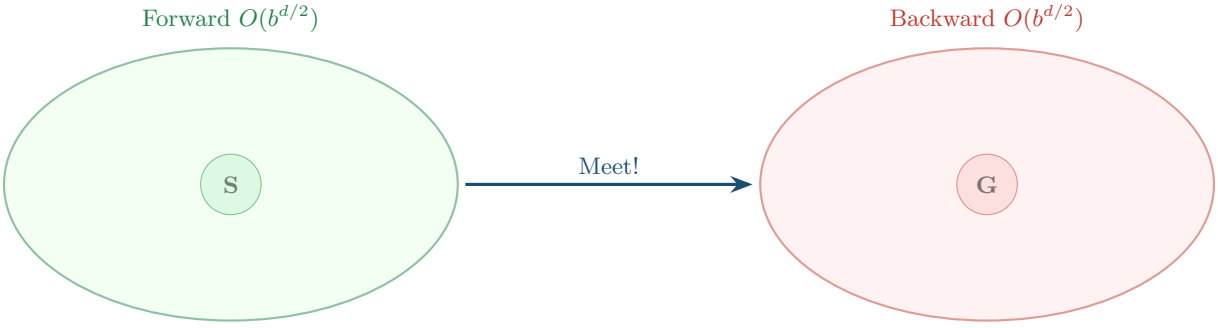


UCS Step-by-Step Trace

Step	Priority Queue (node:g)	Expanded	Notes
0	[(A:0)]	–	Initialize
1	[(C:2),(B:4)]	A(g=0)	C=0+2=2, B=0+4=4
2	[(B:3),(D:10)]	C(g=2)	B via C=2+1=**3**<4: update. D=2+8=10
3	[(D:6),(C:8),(D:10)]	B(g=3)	D via B=3+3=6, C via B=3+5=8
4	[rest...]	D(g=6)	**GOAL! g=6. Optimal path: A→C→B→D**

Comparison: A→B→D=4+3=7. A→C→D=2+8=10. **A→C→B→D=2+1+3=6**. UCS correctly found the optimal.

2.5 1



$b = 10$, $d = 6$: Forward alone $= 10^6 = 1{,}000{,}000$

Bidirectional $= 2 \times 10^3 = 2{,}000$**. 500× speedup!**

2.6 1

Algorithm	Data Structure	Complete?	Optimal?	Time	Space
DFS	Stack (LIFO)	No*	No	$O(b^m)$	$O(bm)$
BFS	Queue (FIFO)	Yes	Yes**	$O(b^d)$	$O(b^d)$
IDS	Stack (repeated)	Yes	Yes**	$O(b^d)$	$O(bd)$
UCS	PQ by $g(n)$	Yes	Yes	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$
Bidir.	Two frontiers	Yes***	Yes***	$O(b^{d/2})$	$O(b^{d/2})$

**DFS complete on finite acyclic graphs with cycle detection. **Optimal for uniform step costs only. ***Under certain conditions (known goal, reversible actions).*

3. 1

Informed search uses a **heuristic function** $h(n)$: an estimate of cost from n to the nearest goal. $h(\text{goal}) = 0$. A good heuristic can dramatically reduce nodes explored from exponential to near-linear.

3.1 1

$f(n) = h(n)$ (ignores actual cost $g(n)$ already incurred)

Greedy always expands the node that *looks* closest to the goal. It is fast but **neither complete nor optimal**.

3.2 1

$f(n) = g(n) + h(n)$

- $g(n)$: exact cost from start to n (known, accumulated)
- $h(n)$: estimated cost from n to goal (heuristic)
- $f(n)$: estimated total path cost through n

A* balances exploration of already-cheap paths (g) vs promising paths to goal (h).

3.2.1 1Romania Map: Greedy vs A* Side-by-Side

h=100; [goalnode] (Bu) [below=1.5cm of Fa] Bu
h=0; [-Stealth,dblue] (Ar) – node[left,black] 140 (Si); [-Stealth,dblue] (Ar) – node[right,black] 118 (Ti); [-Stealth,dblue] (Ar) – node[right,black] 75 (Ze); [-Stealth,dblue] (Si) – node[left,black] 99 (Fa); [-Stealth,dblue] (Si) – node[right,black] 80 (Ri); [-Stealth,dblue] (Fa) – node[left,black] 211 (Bu); [-Stealth,dblue] (Ri) – node[right,black] 97 (Pi); [-Stealth,dblue] (Pi) – node[right,black] 101 (Bu); [dred,ultra thick] (Ar) – (Si) – (Fa) – (Bu); [right=0.3cm of Bu,color=dred] **Greedy: cost=450**; [dgreen,ultra thick,dashed] (Ar) to[bend right=8] (Si); [dgreen,ultra thick,dashed] (Si) to[bend right=8] (Ri) – (Pi) to[bend right=8] (Bu); [above=0.3cm of Pi,color=dgreen] **A*:**

cost=418 (optimal!);

A* Trace – Romania (start

Step	Open List: (node: $g+h=f$)	Expand (lowest f)
1	[(Arad:0+366=366)]	Arad
2	[(Sibiu:140+253=**393**),(Tim:118+329=447),(Zer:75+374=449)]	Sibiu (f=393)
3	[(Rim:220+193=**413**),(Fag:239+176=415),(Tim:447),(Zer:449)]	Rimnicu (f=413)
4	[(Fag:415),(Pit:317+100=**417**),(Tim:447),(Zer:449)]	Fagaras (f=415)
5	[(Pit:**417**),(Buc-Fag:239+211=450),(Tim:447),(Zer:449)]	Pitesti (f=417)
6	[(Buc-Pit:317+101+0=**418**),(Buc-Fag:450),...]	**Bucharest f=418 = GOAL!**

Arad→Sibiu→Fagaras→Bucharest=450 (3 cities, greedy at each step).
A* found Arad→Sibiu→Rimnicu→Pitesti→Bucharest=418 (4 cities but cheaper). A* is optimal.

Greedy found

3.3 1

$h(n) \leq h^*(n)$ for all n **(Admissibility)**

$h(n) \leq c(n, a, n') + h(n')$ **(Consistency / Triangle Inequality)**

Optimality Conditions for A*

Version	Requirement	Why It Works
A* Tree Search	h is **admissible**	Never overestimates ⇒ won't skip optimal path
A* Graph Search	h is **consistent** (monotone)	$f(n)$ non-decreasing along paths ⇒ first dequeue = optimal

Q: [Easy] What is an admissible heuristic? Give an example.
A: $h(n) \leq h^*(n)$: never overestimates the true cost. For 8-puzzle: Manhattan distance is admissible (each tile needs at least that many moves). Euclidean distance is also admissible.

Q: [Medium] 8-puzzle: h_1=Manhattan distance, h_2=Euclidean distance. Which is more informed? Both admissible?
A: Both admissible since actual moves ≥ any geometric distance. $h_1 \geq h_2$ always (Manhattan ≥ Euclidean in grid), so h_1 is more informed. A* with h_1 expands fewer nodes — it "knows more".

Q: [Hard] Prove that consistency implies A* graph search is optimal.
A: Consistency ⇒ $f(n') = g(n') + h(n') \geq g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$. So f is non-decreasing along paths. When A* dequeues node n, all remaining open nodes have $f \geq f(n)$. When goal G is dequeued with cost $g(G)$, since $h(G) = 0$, we have $f(G) = g(G)$. Any other path to G through the open list has $f \geq g(G)$, so no cheaper path exists. **Therefore $g(G)$ is optimal.**

3.4 1

If $h_2(n) \geq h_1(n)$ for all n (and both admissible), then h_2 **dominates** h_1. A* with h_2 will expand fewer or equal nodes. Always prefer the higher admissible heuristic.

4. 1

Two-player zero-sum games: MAX wins exactly what MIN loses. **MAX** maximizes utility; **MIN** minimizes it. Both play optimally.

4.1 1

$$\text{MINIMAX}(n) = \begin{cases} \text{UTILITY}(n) & n \text{ terminal} \\ \max_{c \in \text{children}(n)} \text{MINIMAX}(c) & n \text{ is MAX node} \\ \min_{c \in \text{children}(n)} \text{MINIMAX}(c) & n \text{ is MIN node} \end{cases}$$

Complexity: Time $O(b^m)$; Space $O(bm)$ (DFS traversal).

4.1.1 1Minimax Full Tree Dry Run

$v = 3$ child node[leaf] (D) 3 child node[leaf] (E) 5 child node[minnode] (C) MIN
$v = 2$ child node[leaf] (F) 2 child node[leaf] (G) 9 ; [left=0.3cm of B,font=,color=mgray] $\min(3,5) = 3$; [right=0.3cm of C,font=,color=mgray] $\min(2,9) = 2$; [right=0.4cm of A,font=,color=dblue] $\max(3,2) = \mathbf{3}$; [dgreen,very thick,dashed,-Stealth] (A) – node[left,font=,color=dgreen] **play** (B);

Minimax Value Table

Node	Type	Children values	Value	Reasoning
D	Terminal	–	3	Leaf utility
E	Terminal	–	5	Leaf utility
F	Terminal	–	2	Leaf utility
G	Terminal	–	9	Leaf utility
B	MIN	D=3, E=5	3	$\min(3,5) = 3$ — MIN picks worst for MAX
C	MIN	F=2, G=9	2	$\min(2,9) = 2$
A	MAX	B=3, C=2	**3**	$\max(3,2) = 3$ — MAX plays to B

4.2 1

Alpha-Beta eliminates branches that cannot affect the minimax value. Achieves same result as full minimax with fewer node evaluations.

Alpha and Beta Variables

α **(alpha)**: Best value MAX can *guarantee* from current path. Initialized to $-\infty$. Updated only at MAX nodes.
β **(beta)**: Best value MIN can *guarantee* from current path. Initialized to $+\infty$. Updated only at MIN nodes.

Pruning rules:
At MIN node: if current value $\leq \alpha \Rightarrow$ **prune** (MAX already has better option elsewhere).
At MAX node: if current value $\geq \beta \Rightarrow$ **prune** (MIN already has better option elsewhere).

At MIN node: prune remaining children if $v \leq \alpha$

At MAX node: prune remaining children if $v \geq \beta$

4.2.1 1Alpha-Beta Full Dry Run with Three MAX Children

$v = 3$ child node[minnode] (B) MIN
β=3
$v = 3$ child node[leaf] 3 child node[leaf] 5 child node[minnode] (C) MIN
α=3
$v = 2$ child node[leaf] 2 child[missing] child node[minnode] (D) MIN

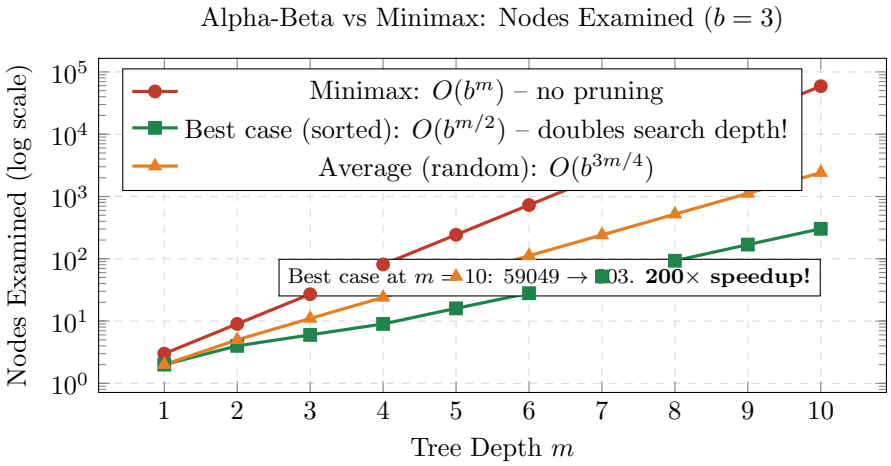
$v = 1$ child node[leaf] 7 child node[leaf] 1 ; [draw=dred,dashed,fill=pink,minimum width=9mm,minimum height=7mm,font=] (pruned) at (7.5,-4.2) 9; [dred,dashed,thick] (C) – (pruned); [right=0.1cm of pruned,font=,color=dred]

PRUNED ($2 \leq \alpha$=3); [dgreen,very thick,dashed,-Stealth] (A) – node[left,font=,color=dgreen] **play to B** (B);

Alpha-Beta Detailed Trace

Step	Node	Action	α	β	Result
1	A(MAX)	Initialize, visit B	$-\infty$	$+\infty$	Call B
2	B(MIN)	Leaf 3: $v = 3$, $\beta = \min(+\infty, 3) = 3$	$-\infty$	3	Partial B=3
3	B(MIN)	Leaf 5: $\min(3, 5) = 3$. B done	$-\infty$	3	B=3
4	A(MAX)	B=3: $\alpha = \max(-\infty, 3) = 3$. Visit C	3	$+\infty$	$\alpha = 3$
5	C(MIN)	Leaf 2: $v = 2$. Is $v \leq \alpha$? $2 \leq 3$?	3	$+\infty$	**YES! Prune child 9. C=2**
6	A(MAX)	C=2: $\max(3, 2) = 3$. Visit D	3	$+\infty$	α stays 3
7	D(MIN)	Leaf 7: $\beta = \min(+\infty, 7) = 7$	3	7	D partial=7
8	D(MIN)	Leaf 1: $\min(7, 1) = 1$. D=1	3	7	D=1
9	A(MAX)	$\max(3, 2, 1) = 3$. Done	3	$+\infty$	**Final=3. Play to B.**

4.2.2 1Alpha-Beta Best/Worst/Average Case Graph



Best case *(successors sorted best-first): only* $O(b^{m/2})$ *nodes – doubles effective search depth within same budget.* ***Worst case*** *(sorted worst-first):* $O(b^m)$ *– no pruning at all.* ***Move ordering*** *is critical for alpha-beta effectiveness.*

Q: [Easy] What do α and β represent?
A: α=best guaranteed value for MAX; β=best guaranteed value for MIN. Prune at MIN when $v \leq \alpha$; at MAX when $v \geq \beta$.

Q: [Medium] Full minimax examines 10^9 nodes. With optimal alpha-beta?
A: Best case: $O(b^{m/2}) = (b^m)^{0.5} = (10^9)^{0.5} \approx 31{,}623$ nodes. About **31,600**× reduction.

Q: [Hard] A(MAX)→B(MIN),C(MIN). B has leaves 6,4,9; C has leaves 2,8,3. Trace alpha-beta with $\alpha = -\infty$, $\beta = +\infty$. Which nodes are pruned?
A: B: leaf 6→ $\beta = 6$; leaf 4→ $\beta = 4$; leaf 9: $\min(4, 9) = 4$. B=4. Back to A: $\alpha = \max(-\infty, 4) = 4$. C: leaf 2→ $v = 2$. Check $v \leq \alpha$? $2 \leq 4$? **YES ⇒ prune leaves 8 and 3**. C=2. A=$\max(4, 2) = 4$. Play to B. **Pruned: nodes 8 and 3 under C.**

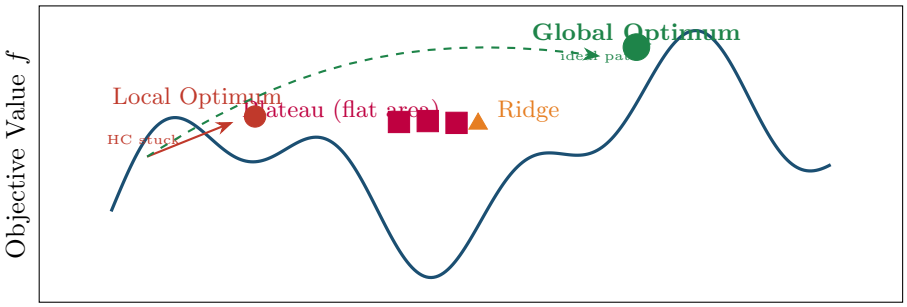
5. 1

Local search works on a **single current state** with no frontier or path memory. It moves to neighbors to improve an **objective function** $f(n)$.

Advantages: Uses very little memory $O(1)$; works for continuous/very large spaces.
Disadvantages: Can get stuck at local optima, plateaus, or ridges.

5.0.1 1Objective Function Landscape – Problems Illustrated

Local Search Landscape: Local Optima, Plateaus, Ridges, Global Optimum

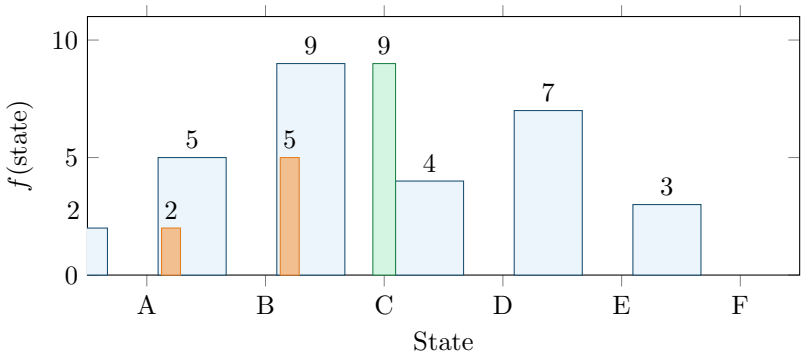


Hill Climbing Variants Compared

Variant	How It Works	Pros / Cons
Simple HC	Move to FIRST improving neighbor	Fast per step; may miss better neighbors
Steepest Ascent HC	Evaluate ALL neighbors; move to BEST	Better quality; still stuck at local optima
Stochastic HC	Pick randomly among uphill moves	Less trapping; slower convergence
Random Restart HC	Repeat from random start states	Probabilistically complete
Simulated Annealing	Accept downhill with prob $e^{\Delta E/T}$, $T \to 0$	Escapes local optima; guaranteed global opt

5.0.2 1Steepest Ascent HC Dry Run with Graph

Objective Values by State: Steepest Ascent Trace



Steepest Ascent HC: Start

Step	Current	f	Neighbors	f(neighbors)	Move to
1	A	2	B	f(B)=5 > 2	B
2	B	5	A, C	f(A)=2, f(C)=9	C (best)
3	C	9	B, D	f(B)=5, f(D)=4	**Both < 9: LOCAL (=GLOBAL) OPTI-MUM**

Path: A→B→C.

Found global optimum in 2 steps. (Convenient here; usually local optima block progress.)

Simulated Annealing: Escape Local Optima

Accept worse states with probability $P = e^{\Delta E/T}$ where $\Delta E = f(\text{new}) - f(\text{current}) < 0$ and T is temperature (decreasing).

When T is high (early): $P \approx 1$ – accept almost any move (explore).
When $T \to 0$ (later): $P \to 0$ – only accept improvements (exploit).

Cooling schedule: $T(t) = T_0 \cdot \alpha^t$ (geometric) or $T_0 / \log(t+1)$ (logarithmic). Logarithmic cooling guarantees convergence to global optimum.

Q: [Easy] What is a local optimum? Why does HC get stuck?
A: A local optimum is a state where all neighbors have lower f value but it is not the global maximum. HC gets stuck because it only moves to neighbors with higher f, so it cannot escape.

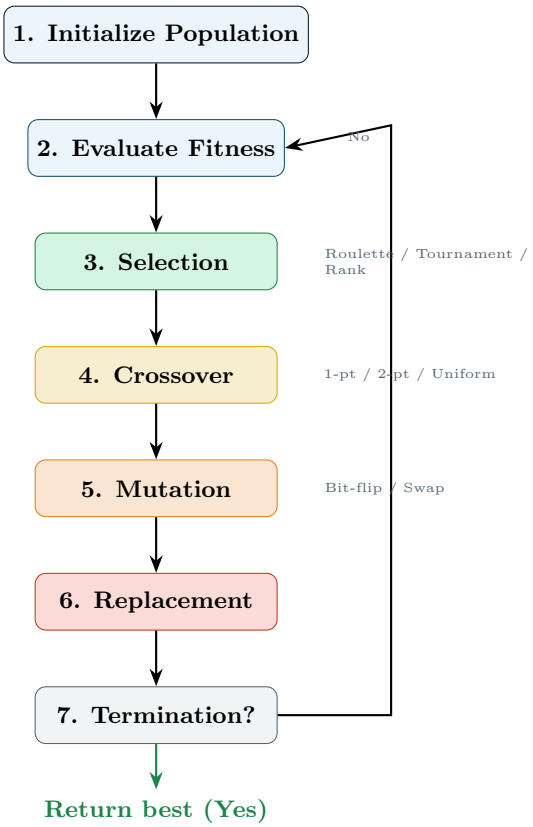
Q: [Medium] States: A(4)↔B(7)↔C(5)↔D(9)↔E(3). Start=A. Steepest ascent HC. Does it find global?
A: A(4)→B(7). At B: neighbors A(4)<7 and C(5)<7. **HC stops at B(7)**. D(9) is the global optimum but HC can't reach it without going down through C(5). HC fails here.

Q: [Hard] Why is random-restart HC probabilistically complete? If $p = 0.2$, expected restarts?
A: Each restart independently finds the global optimum with probability $p > 0$. Probability of never succeeding in k restarts $= (1-p)^k \to 0$. Expected restarts $= 1/p = 1/0.2 = 5$. It is probabilistically complete (probability of finding optimum approaches 1 as restarts increase).

6. 1

GAs maintain a **population** of candidate solutions. Evolution via **selection**, **crossover**, and **mutation** over many generations.

6.0.1 1GA Cycle Diagram



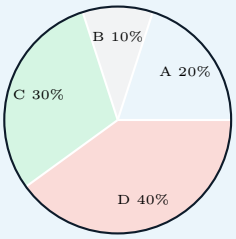
6.1 1

6.1.1 1Roulette Wheel (Fitness Proportionate) Selection

$$P(i) = \frac{\text{fitness}(i)}{\sum_{j=1}^{N} \text{fitness}(j)}$$

Roulette Wheel Dry Run

Population: A(4), B(2), C(6), D(8). Total fitness=20.
$P(A) = 4/20 = 0.20$, $P(B) = 2/20 = 0.10$, $P(C) = 6/20 = 0.30$, $P(D) = 8/20 = 0.40$



Cumulative ranges:
A: $[0.00, 0.20)$
B: $[0.20, 0.30)$
C: $[0.30, 0.60)$
D: $[0.60, 1.00)$

$r = 0.45 \rightarrow$ **C selected**
$r = 0.72 \rightarrow$ **D selected**
$r = 0.18 \rightarrow$ **A selected**

6.1.2 1Tournament Selection

Tournament Selection (k

Pick k individuals at random; the one with highest fitness wins.
Tournament 1: randomly pick C(6) and A(4). Winner: **C**(6).
Tournament 2: randomly pick D(8) and B(2). Winner: **D**(8).
Parents for crossover: C and D. Higher $k \Rightarrow$ more selection pressure.

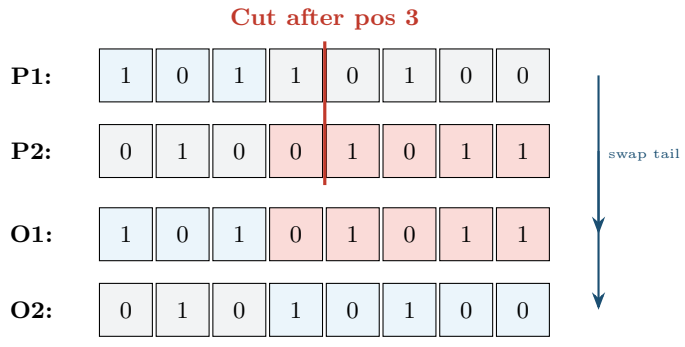
6.1.3 1Rank-Based Selection

$$P(\text{rank } r) = \frac{r}{N(N+1)/2}$$

Sort individuals by fitness; assign rank 1 (worst) to N (best). Avoids domination by super-fit individuals.

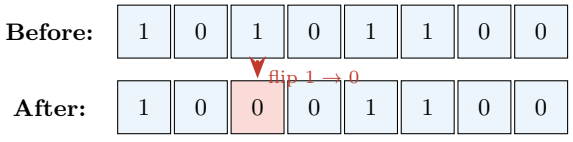
6.2 1

6.2.1 1Single-Point Crossover



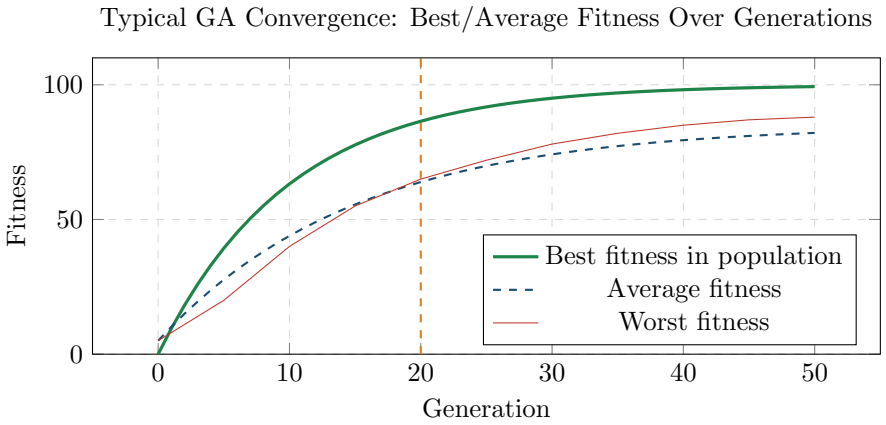
O1=P1 head + P2 tail. O2=P2 head + P1 tail.

6.2.2 1Bit-Flip Mutation



Mutation probability p_m (typically 0.001 to 0.01) applied to each bit independently.

6.2.3 1GA Fitness Over Generations Graph



Q: [Medium] Population A(10), B(20), C(30), D(40). Roulette probabilities and cumulative ranges?
A: Total=100. $P(A)$=10%: [0.00,0.10). $P(B)$=20%: [0.10,0.30). $P(C)$=30%: [0.30,0.60). $P(D)$=40%: [0.60,1.00). $r = 0.55 \rightarrow$C. $r = 0.85 \rightarrow$D.

Q: [Hard] P1=110010, P2=001101. 2-point crossover after positions 2 and 4. Then bit-flip O1 at position 5.
A: Segments: P1=[11—00—10], P2=[00—11—01]. Swap middle segment: O1=11+11+10=**111110**. O2=00+00+01=**000001**. Mutation O1 position 5: flip 1→0: **111100**.

7. 1

Types of Machine Learning

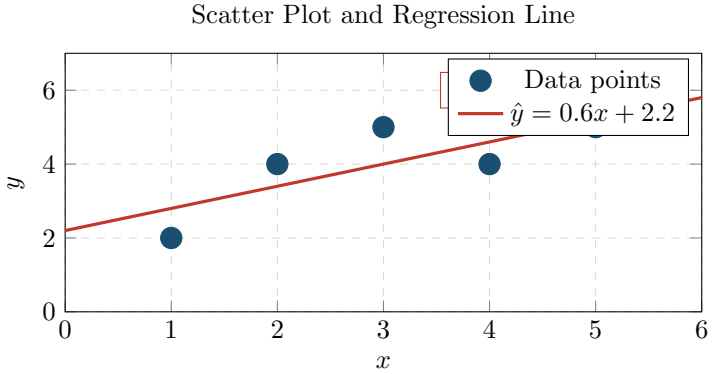
Type	Description	Examples
Supervised	Labeled data: learn $f : X \rightarrow Y$	Classification, Regression
Unsupervised	Find structure in unlabeled data	Clustering, Dim. Reduction
Reinforcement	Agent learns from environment rewards	Game playing, Robotics

$y = w_1 x + w_0$ **(Simple Linear Regression)**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$w_1 = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}, \qquad w_0 = \bar{y} - w_1 \bar{x}$$

7.0.1 1Linear Regression Dry Run with Scatter Plot



Computation Table: x

, y=[2,4,5,4,5]]

x_i	y_i	x_i^2	$x_i y_i$
1	2	1	2
2	4	4	8
3	5	9	15
4	4	16	16
5	5	25	25
15	**20**	**55**	**66**

$$w_1 = \frac{5(66) - 15(20)}{5(55) - 15^2} = \frac{330 - 300}{275 - 225} = \frac{30}{50} = \mathbf{0.6}$$

$\bar{x} = 15/5 = 3$, $\bar{y} = 20/5 = 4$

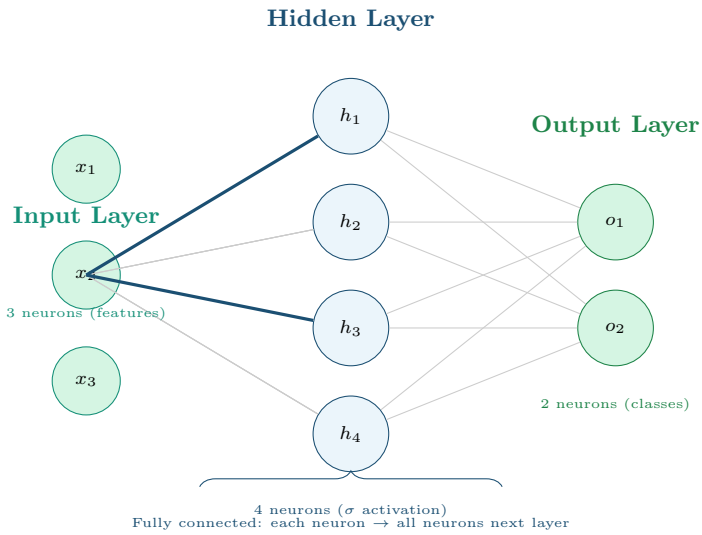
$w_0 = 4 - 0.6(3) = 4 - 1.8 = \mathbf{2.2}$

$\hat{y} = 0.6x + 2.2$

Check: $\hat{y}(1) = 2.8$, $\hat{y}(3) = 4.0$, $\hat{y}(5) = 5.2$

8. 1

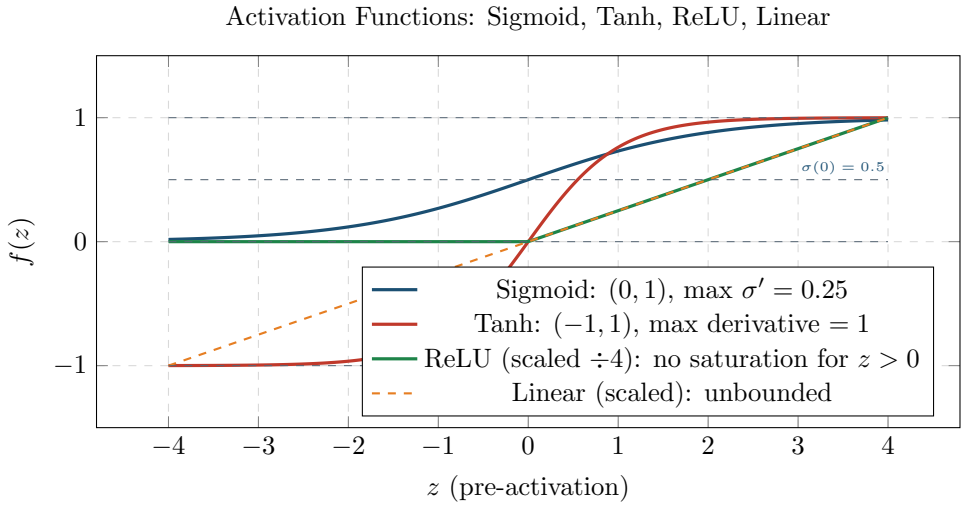
8.1 1



Key facts: Each connection has a weight. Each neuron (except input) has a bias. Learning = adjusting weights and biases via backpropagation. More hidden layers = deeper network = more complex features.

8.2 1

8.2.1 1All Major Activation Functions



Sigmoid in Detail – Used in Backpropagation

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \leq 0.25$$

Key values: $\sigma(0) = 0.5$, $\sigma(1) \approx 0.731$, $\sigma(-1) \approx 0.269$, $\sigma(2) \approx 0.880$.
Maximum derivative: $\sigma'(0) = 0.5 \times 0.5 = 0.25$ (at $z = 0$).
Vanishing gradient: for $|z| > 3$, $\sigma' \approx 0$, gradients shrink and deep layers learn very slowly.

Activation Function Comparison Table

Function	Formula	Range	Properties & Issues
Sigmoid	$1/(1+e^{-z})$	$(0,1)$	Smooth; probabilistic output; saturates at extremes; max $\sigma' \leq 0.25$; vanishing gradient in deep nets
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$(-1,1)$	Zero-centered (better gradient flow than sigmoid); still saturates
ReLU	$\max(0, z)$	$[0,\infty)$	No saturation for $z > 0$; fast to compute; dying ReLU: neurons with $z < 0$ always output 0
Leaky ReLU	z if $z > 0$, $0.01z$ otherwise	$(-\infty,\infty)$	Fixes dying ReLU; small gradient for $z < 0$
Softmax	$e^{z_k} / \sum_j e^{z_j}$	$(0,1)$	Multi-class output layer; outputs sum to 1 (probability distribution)

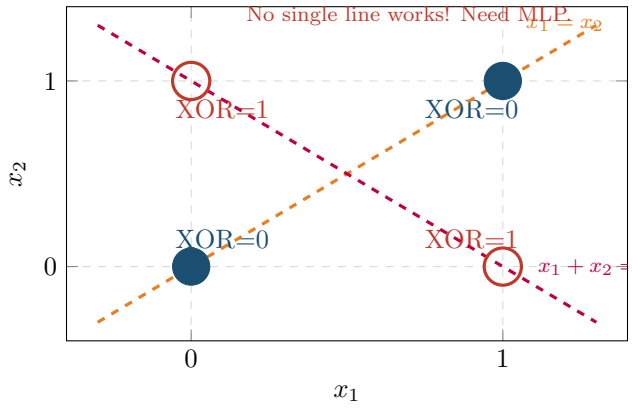
8.3 1

A single perceptron computes $\hat{y} = \text{step}(\mathbf{w} \cdot \mathbf{x} + b)$. The decision boundary is a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$.

$$\hat{y} = \text{step}(\sum_i w_i x_i + b), \quad \text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i, \qquad b \leftarrow b + \eta(y - \hat{y}) \qquad \textbf{(Perceptron Update)}$$

XOR is NOT Linearly Separable (no line can separate ∘ from •)



AND and OR are linearly separable (single perceptron works). XOR requires at least one hidden layer. This was a major result in AI history (Minsky & Papert, 1969) that temporarily halted ANN research.

9. 1

Backpropagation applies the **chain rule** of calculus to compute gradients of the loss with respect to every weight. We then do **gradient descent** to minimize loss.

Core Notation

$z_j^{(l)}$: pre-activation at neuron j, layer l $\quad (z = \sum_i w_{ij} a_i + b_j)$
$a_j^{(l)} = \sigma(z_j^{(l)})$: post-activation (output of neuron)
$\delta_j^{(l)} = \partial L / \partial z_j^{(l)}$: local gradient / error signal (delta)
Loss: $L = \frac{1}{2}(y - a_{\text{out}})^2$ (MSE, factor $\frac{1}{2}$ for clean derivative)
Update: $w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$ $\quad (\eta = \text{learning rate})$

$$\delta_{\text{out}} = (a_{\text{out}} - y) \cdot \sigma'(z_{\text{out}}) \qquad \textbf{(Output delta)}$$

$$\delta_j^{(l)} = \sigma'(z_j^{(l)}) \cdot \sum_k \delta_k^{(l+1)} \cdot w_{jk} \qquad \textbf{(Hidden delta, chain rule)}$$

$$\frac{\partial L}{\partial w_{ij}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \qquad \textbf{(Gradient for weight } w_{ij}\textbf{)}$$

9.1 1

9.1.1 1Network Architecture Diagram

Step 1: Forward Pass

Hidden layer pre-activations:

$$z_{h1} = w_{11}x_1 + w_{21}x_2 + b_h = 0.15(0.05) + 0.25(0.10) + 0.35 = 0.0075 + 0.025 + 0.35 = \mathbf{0.3825}$$
$$z_{h2} = w_{12}x_1 + w_{22}x_2 + b_h = 0.20(0.05) + 0.30(0.10) + 0.35 = 0.010 + 0.030 + 0.35 = \mathbf{0.3900}$$

Hidden layer activations (Sigmoid):

$$a_{h1} = \sigma(0.3825) = \frac{1}{1 + e^{-0.3825}} = \frac{1}{1 + 0.6820} = \frac{1}{1.6820} = \mathbf{0.5944}$$
$$a_{h2} = \sigma(0.3900) = \frac{1}{1 + e^{-0.3900}} = \frac{1}{1 + 0.6771} = \frac{1}{1.6771} = \mathbf{0.5963}$$

Output layer:

$$z_o = w_{1o} \cdot a_{h1} + w_{2o} \cdot a_{h2} + b_o = 0.40(0.5944) + 0.45(0.5963) + 0.60$$
$$= 0.23776 + 0.26834 + 0.60 = \mathbf{1.1061}$$
$$a_o = \sigma(1.1061) = \frac{1}{1 + e^{-1.1061}} = \frac{1}{1 + 0.3307} = \frac{1}{1.3307} = \mathbf{0.7514}$$

Loss: $L = \frac{1}{2}(y - a_o)^2 = \frac{1}{2}(0.01 - 0.7514)^2 = \frac{1}{2}(-0.7414)^2 = \frac{1}{2}(0.5497) = \mathbf{0.2748}$

Step 2: Backward Pass – Output Layer

Sigmoid derivative at output:

$$\sigma'(z_o) = a_o(1 - a_o) = 0.7514 \times 0.2486 = \mathbf{0.1868}$$

Output delta (chain rule: $\partial L/\partial z_o = \partial L/\partial a_o \cdot \partial a_o/\partial z_o$):

$$\frac{\partial L}{\partial a_o} = a_o - y = 0.7514 - 0.01 = 0.7414$$
$$\delta_o = \frac{\partial L}{\partial a_o} \cdot \sigma'(z_o) = 0.7414 \times 0.1868 = \mathbf{0.1385}$$

Gradients for output weights:

$$\frac{\partial L}{\partial w_{1o}} = \delta_o \cdot a_{h1} = 0.1385 \times 0.5944 = \mathbf{0.08232}$$
$$\frac{\partial L}{\partial w_{2o}} = \delta_o \cdot a_{h2} = 0.1385 \times 0.5963 = \mathbf{0.08258}$$
$$\frac{\partial L}{\partial b_o} = \delta_o \cdot 1 = \mathbf{0.1385}$$

Step 3: Backward Pass – Hidden Layer

Propagate delta back through output weights:

$$\frac{\partial L}{\partial a_{h1}} = \delta_o \cdot w_{1o} = 0.1385 \times 0.40 = 0.05540$$
$$\frac{\partial L}{\partial a_{h2}} = \delta_o \cdot w_{2o} = 0.1385 \times 0.45 = 0.06233$$

Sigmoid derivatives at hidden layer:

$$\sigma'(z_{h1}) = a_{h1}(1 - a_{h1}) = 0.5944 \times 0.4056 = \mathbf{0.2410}$$
$$\sigma'(z_{h2}) = a_{h2}(1 - a_{h2}) = 0.5963 \times 0.4037 = \mathbf{0.2407}$$

Hidden deltas:

$$\delta_{h1} = \frac{\partial L}{\partial a_{h1}} \cdot \sigma'(z_{h1}) = 0.05540 \times 0.2410 = \mathbf{0.013351}$$
$$\delta_{h2} = \frac{\partial L}{\partial a_{h2}} \cdot \sigma'(z_{h2}) = 0.06233 \times 0.2407 = \mathbf{0.015015}$$

Gradients for hidden weights:

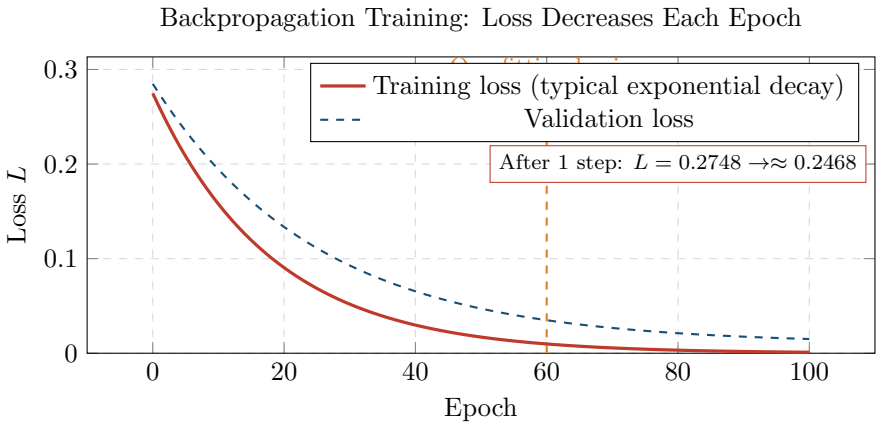
$$\frac{\partial L}{\partial w_{11}} = \delta_{h1} \cdot x_1 = 0.013351 \times 0.05 = \mathbf{0.000668} \qquad \frac{\partial L}{\partial w_{21}} = \delta_{h1} \cdot x_2 = 0.013351 \times 0.10 = \mathbf{0.001335}$$
$$\frac{\partial L}{\partial w_{12}} = \delta_{h2} \cdot x_1 = 0.015015 \times 0.05 = \mathbf{0.000751} \qquad \frac{\partial L}{\partial w_{22}} = \delta_{h2} \cdot x_2 = 0.015015 \times 0.10 = \mathbf{0.001502}$$

Step 4: Weight Updates (η

$$w_{1o} \leftarrow 0.40 - 0.5(0.08232) = 0.40 - 0.04116 = \mathbf{0.35884}$$
$$w_{2o} \leftarrow 0.45 - 0.5(0.08258) = 0.45 - 0.04129 = \mathbf{0.40871}$$
$$b_o \leftarrow 0.60 - 0.5(0.1385) = 0.60 - 0.06925 = \mathbf{0.53075}$$

$$w_{11} \leftarrow 0.15 - 0.5(0.000668) = 0.15 - 0.000334 = \mathbf{0.149666}$$
$$w_{21} \leftarrow 0.25 - 0.5(0.001335) = 0.25 - 0.000668 = \mathbf{0.249332}$$
$$w_{12} \leftarrow 0.20 - 0.5(0.000751) = 0.20 - 0.000376 = \mathbf{0.199624}$$
$$w_{22} \leftarrow 0.30 - 0.5(0.001502) = 0.30 - 0.000751 = \mathbf{0.299249}$$

9.1.2 1Loss Reduction Over Training Epochs



After the one weight update above, loss drops from 0.2748 to ≈0.2468. Training repeats this cycle for many epochs (typically hundreds to thousands) until convergence.

9.2 1

$$\delta_j^{(l)} = \sigma'(z_j^{(l)}) \cdot \sum_k \delta_k^{(l+1)} \cdot w_{jk}^{(l+1)} \quad \textbf{(General delta – chain rule)}$$

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \quad \textbf{(Gradient for any weight)}$$

The key insight: delta at layer l depends on deltas at layer $l+1$ (already computed), multiplied by the connecting weights and the local sigmoid derivative. This is why we go **backward** from output to input.

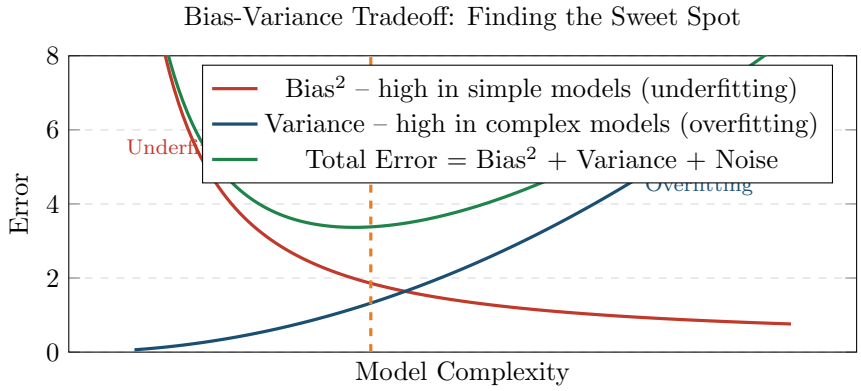
Q: [Easy] What is $\sigma'(z)$ for sigmoid? If $\sigma(z) = 0.7$, compute $\sigma'(z)$.
A: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. If $\sigma(z) = 0.7$: $\sigma' = 0.7 \times 0.3 = 0.21$.

Q: [Medium] Why multiply by $\sigma'(z)$ when computing δ?
A: By the chain rule: $\delta = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} = \frac{\partial L}{\partial a} \cdot \sigma'(z)$. The $\sigma'(z)$ term measures how much the pre-activation z affects the output a. Without it, we'd have the wrong gradient.

Q: [Hard] Network: $x = 0.6$, 1 hidden ($w_h = 0.5$, $b_h = 0$), 1 output ($w_o = 0.8$, $b_o = 0$). $y = 1$, $\eta = 0.1$. Full forward and backward pass.
A: **Forward**: $z_h = 0.5(0.6) = 0.3$, $a_h = \sigma(0.3) = 0.5744$. $z_o = 0.8(0.5744) = 0.4595$, $a_o = \sigma(0.4595) = 0.6129$. $L = \frac{1}{2}(1 - 0.6129)^2 = 0.0750$.
Backward: $\sigma'(z_o) = 0.6129(0.3871) = 0.2373$. $\delta_o = (0.6129 - 1)(0.2373) = (-0.3871)(0.2373) = -0.09189$. $\partial L/\partial w_o = \delta_o \cdot a_h = -0.09189(0.5744) = -0.05279$. $w_o \leftarrow 0.8 - 0.1(-0.05279) = \mathbf{0.80528}$.
$\sigma'(z_h) = 0.5744(0.4256) = 0.2445$. $\delta_h = (\delta_o \cdot w_o) \cdot \sigma'(z_h) = (-0.09189 \times 0.8)(0.2445) = -0.07351 \times 0.2445 = -0.017972$. $\partial L/\partial w_h = \delta_h \cdot x = -0.017972(0.6) = -0.010783$. $w_h \leftarrow 0.5 - 0.1(-0.010783) = \mathbf{0.501078}$.

10. 1

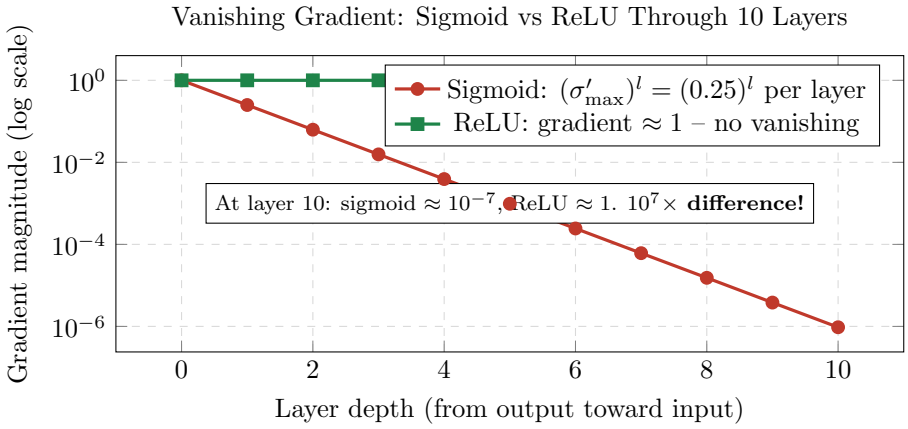
10.1 1



Underfitting vs Good Fit vs Overfitting

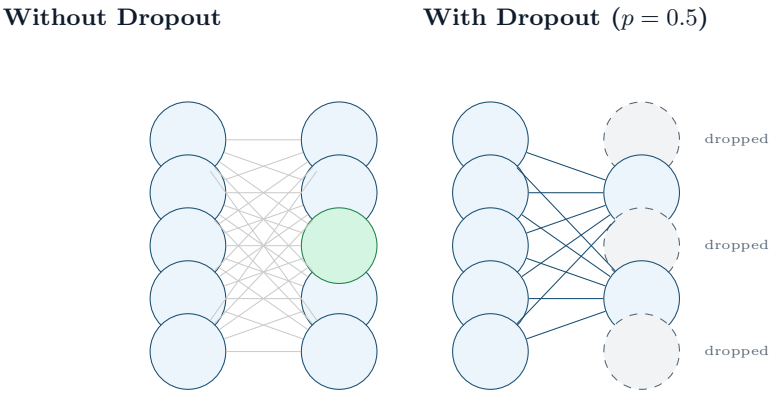
Condition	Description	Train Err	Test Err	Solutions
Underfitting	Model too simple; misses patterns	High	High	More complexity, more features, fewer constraints
Good Fit	Captures true signal	Low	Low	–
Overfitting	Memorizes noise; no generalization	Very Low	High	Regularization (L_1/L_2), Dropout, Early stopping, More training data

10.2 1



Each sigmoid layer multiplies gradients by at most $\sigma' \leq 0.25$. After 10 layers: $0.25^{10} \approx 10^{-6}$. Early layers receive almost zero gradient and barely learn. This is why deep networks use ReLU instead of sigmoid in hidden layers.

10.3 1



11. 1

11.1 1

		Predicted	
		Positive	**Negative**
Actual	**Positive**	**TP** True Positive	**FN** False Negative (Type II)
	Negative	**FP** False Positive (Type I)	**TN** True Negative

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{“Of all predicted positive, how many actually are?”}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad \text{“Of all actual positive, how many did we catch?”}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

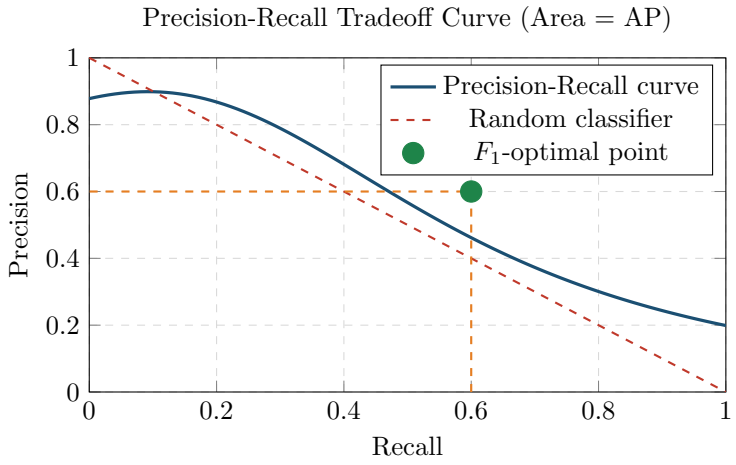
11.1.1 1Numeric Dry Run

Example: 10 samples (5 positive

		Predicted	
		Positive	Negative
Actual	Positive	TP=3	FN=2
	Negative	FP=2	TN=3

Accuracy $= \frac{3+3}{10} = 0.60$ Precision $= \frac{3}{3+2} = 0.60$ Recall $= \frac{3}{3+2} = 0.60$ $F_1 = \frac{2(0.6)(0.6)}{0.6+0.6} = 0.60$

11.1.2 1Precision vs Recall Trade-off



11.2 1

3-Class Confusion Matrix (Cat / Dog / Bird

Actual \ Pred	Cat	Dog	Bird
Cat (10)	8	1	1
Dog (10)	2	7	1
Bird (10)	0	2	8

Class	TP	FP	FN	Prec
Cat	8	2	2	0.800
Dog	7	3	3	0.700
Bird	8	2	2	0.800

Macro Average: Average per-class metrics (treats each class equally).
Macro Prec $= (0.800 + 0.700 + 0.800)/3 = 0.767$. Macro Rec $= 0.767$. Macro $F_1 = 0.767$.

Micro Average: Aggregate all TP/FP/FN across classes (treats each sample equally).
$TP_{\text{tot}} = 8 + 7 + 8 = 23$, $FP_{\text{tot}} = 2 + 3 + 2 = 7$, $FN_{\text{tot}} = 2 + 3 + 2 = 7$.
Micro Prec $= 23/30 = 0.767$. Micro Rec $= 23/30 = 0.767$. Micro $F_1 = 0.767$.

Macro treats all classes equally; use when classes are equally important. Micro is dominated by larger classes.

Q: [Easy] What does FN mean in cancer screening?
A: Patient HAS cancer but model says they DON'T. A False Negative (Type II error / missed diagnosis) is the most dangerous error in medical screening.

Q: [Medium] When prefer Recall over Precision?
A: When missing a true positive (FN) is more costly than a false alarm (FP). Examples: cancer screening (missing cancer is catastrophic), fraud detection (missing fraud is costly). Prefer Precision when false alarms are costly (e.g., spam filtering — don't want valid emails flagged).

Q: [Hard] 4-class matrix rows: C1=[50,2,3,0], C2=[1,40,4,2], C3=[2,3,45,1], C4=[0,1,2,48]. Compute Macro F_1. Which class is worst?
A: TP=[50,40,45,48]. FP(col sum − TP): C1: 3, C2: 6, C3: 9, C4: 3. FN(row sum − TP): C1: 5, C2: 7, C3: 6, C4: 3. P: 50/53=0.943, 40/46=0.870, 45/54=0.833, 48/51=0.941. R: 50/55=0.909, 40/47=0.851, 45/51=0.882, 48/51=0.941. F_1: 0.926, 0.860, 0.857, 0.941. Macro $F_1 = (0.926 + 0.860 + 0.857 + 0.941)/4 = 0.896$. **Poorest: C3** ($F_1 = 0.857$).

12. 1

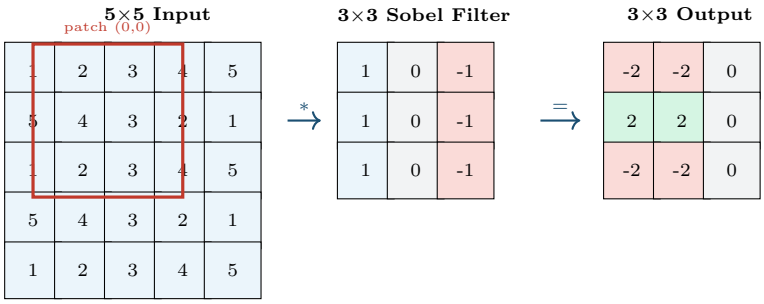
$$\text{Output size} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

N=input size, F=filter/kernel size, P=zero-padding, S=stride.

$$\text{Parameters per conv layer} = (F \times F \times C_{\text{in}} + 1) \times \text{num\_filters}$$

The +1 is for the bias of each filter.

12.1 1

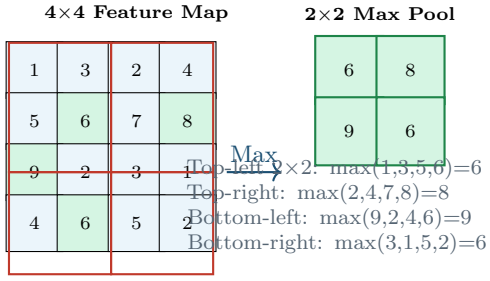


Computing output[0][0] (top-left 3×3 patch of input convolved with filter):

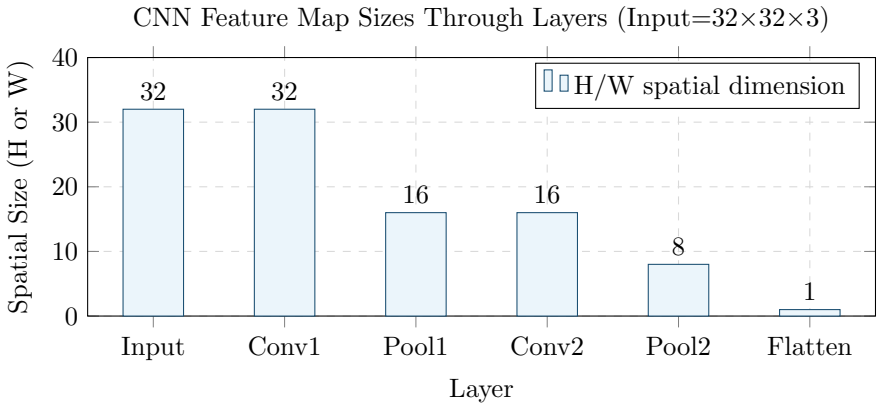
$$\text{out}[0][0] = 1(1)+2(0)+3(-1)+5(1)+4(0)+3(-1)+1(1)+2(0)+3(-1) = (1-3)+(5-3)+(1-3) = -2+2-2 = \mathbf{-2}$$

Output size: $\lfloor (5-3+0)/1 \rfloor + 1 = 3$. So 3×3 output from 5×5 input with 3×3 filter, $P = 0$, $S = 1$.

12.2 1



12.2.1 1Full CNN Architecture Sizes and Parameters



CNN Architecture: Input 32×32×3

Layer	Output Size	Formula	Parameters
Input	$32 \times 32 \times 3$	–	0
Conv1: 16f, 5×5, P=2, S=1	$32 \times 32 \times 16$	$(32-5+4)/1+1=32$	$(5 \times 5 \times 3+1) \times 16 = 1216$
MaxPool1: 2×2, S=2	$16 \times 16 \times 16$	$32/2=16$	0
Conv2: 32f, 3×3, P=1, S=1	$16 \times 16 \times 32$	$(16-3+2)/1+1=16$	$(3 \times 3 \times 16+1) \times 32 = 4640$
MaxPool2: 2×2, S=2	$8 \times 8 \times 32$	$16/2=8$	0
Flatten	2048	$8 \times 8 \times 32 = 2048$	0
Total params	–	–	**5856**

Q: [Easy] Input 32×32, filter 5×5, P=0, S=1. Output size?
A: $(32-5+0)/1+1=28$. Output is 28×28.

Q: [Medium] Why does max pooling give translation invariance?
A: The max over a region is unchanged by small shifts of the feature within that region. An edge detector firing slightly left or right gives the same pooled output — the network doesn't care exactly where the feature is, just that it's present.

Q: [Hard] Input 64×64×3. Conv: 32 filters, 3×3, P=1, S=1. Pool: 2×2, S=2. Conv: 64 filters, 3×3, P=1, S=1. FC: 512 neurons. FC: 10 (output). Compute all sizes and total parameters.
A: Conv1: $(64-3+2)/1+1=64$. Out=$64 \times 64 \times 32$. Params=$(3 \times 3 \times 3+1) \times 32 = 896$. Pool: $32 \times 32 \times 32$. Conv2: $(32-3+2)/1+1=32$. Out=$32 \times 32 \times 64$. Params=$(3 \times 3 \times 32+1) \times 64 = 18496$. Pool2: $16 \times 16 \times 64$. Flatten=16384. FC1: $16384 \times 512 + 512 = 8,389,120$. FC2: $512 \times 10 + 10 = 5130$. **Total: 8,413,642 parameters.**

13. 1

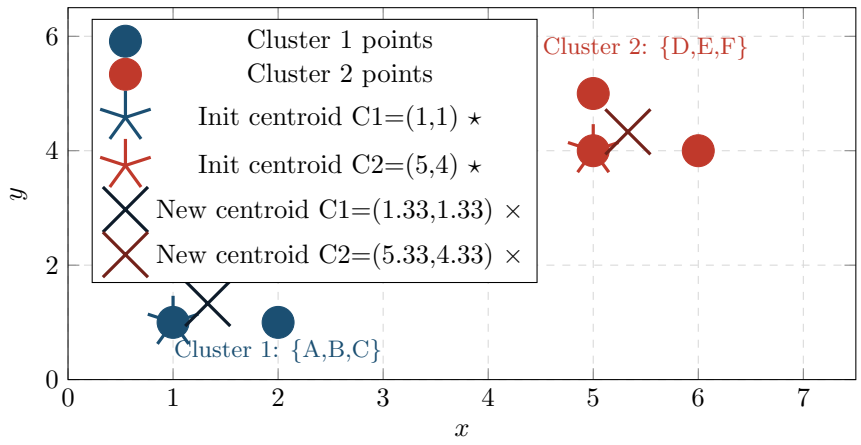
13.1 1

K-Means Algorithm

1. Initialize k centroids randomly (or K-Means++ initialization)
2. **Assignment step**: Assign each point to nearest centroid ($\arg\min_c \|x-\mu_c\|^2$)
3. **Update step**: Recompute centroids as mean of assigned points
4. Repeat steps 2–3 until assignments stop changing (convergence)

Objective: Minimize within-cluster sum of squares (WCSS) = $\sum_c \sum_{x \in c} \|x-\mu_c\|^2$.

K-Means: $k=2$, 2 Iterations to Convergence



K-Means Full Trace: k

Initial centroids: C1=(1,1), C2=(5,4).

Iteration 1 – Assignment (Euclidean distance $d = \sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$):

Point	Dist to C1=(1,1)	Dist to C2=(5,4)	Assign
A(1,1)	0.00	5.00	C1
B(1,2)	1.00	4.47	C1
C(2,1)	1.00	4.24	C1
D(5,4)	5.00	0.00	C2
E(5,5)	5.66	1.00	C2
F(6,4)	5.83	1.00	C2

Update centroids: $C1 = (\frac{1+1+2}{3}, \frac{1+2+1}{3}) = (\frac{4}{3}, \frac{4}{3}) = (1.33, 1.33)$ $C2 = (\frac{5+5+6}{3}, \frac{4+5+4}{3}) = (\frac{16}{3}, \frac{13}{3}) = (5.33, 4.33)$

Iteration 2: Re-assign with new centroids. All points remain in same clusters ⇒ **CONVERGED!**
Final clusters: $\{A, B, C\}$ and $\{D, E, F\}$.
WCSS $= d(A,C1)^2 + d(B,C1)^2 + d(C,C1)^2 + d(D,C2)^2 + d(E,C2)^2 + d(F,C2)^2 = (0.22 + 0.56 + 0.56 + 0.22 + 0.56 + 0.44) = 2.56$

13.2 1

Linkage Criteria

Linkage	Formula	Behavior
Single (MIN)	$\min_{a\in A, b\in B} d(a,b)$	Chaining effect; elongated clusters
Complete (MAX)	$\max_{a\in A, b\in B} d(a,b)$	Compact, roughly equal-sized clusters
Average (UPGMA)	$\frac{1}{\lvert A\rvert\lvert B\rvert}\sum_{a,b} d(a,b)$	Balance between single and complete
Ward's	Minimizes WCSS increase	Best overall; minimizes variance

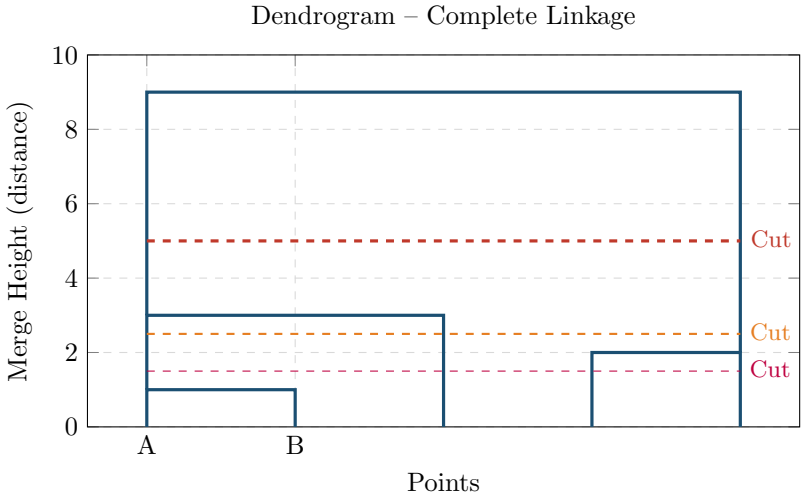
13.2.1 1Complete-Linkage Agglomerative Dry Run

5 Points: A

Initial distance matrix:

	A	B	C	D	E
A	0	1	3	7	9
B	1	0	2	6	8
C	3	2	0	4	6
D	7	6	4	0	2
E	9	8	6	2	0

Step	Action (merge min distance pair)	Clusters	Height
1	Merge A+B (d=1, minimum)	{AB}, {C}, {D}, {E}	1
2	Merge D+E (d=2)	{AB}, {C}, {DE}	2
3	Merge AB+C: complete linkage $\max(d(A,C), d(B,C)) = \max(3,2) = 3$	{ABC}, {DE}	3
4	Merge ABC+DE: $\max(d(A,D), d(A,E), d(B,D), d(B,E), d(C,D), d(C,E)) = \max(7,9,6,8,4,6) = 9$	{ABCDE}	9



Q: [Easy] What are the two steps of K-Means per iteration?
A: **Assignment**: assign each point to nearest centroid. **Update**: recompute centroids as mean of assigned points.

Q: [Medium] Points P1(2,3), P2(5,4), P3(1,2), P4(6,5). $k = 2$, C1=(2,3), C2=(6,5). One iteration.
A: P1: to C1=0, to C2=$\sqrt{25} = 5$ →C1. P2: to C1=$\sqrt{10} = 3.16$, to C2=$\sqrt{2} = 1.41$ →C2. P3: to C1=$\sqrt{2} = 1.41$, to C2=$\sqrt{34} = 5.83$ →C1. P4: to C1=$\sqrt{25} = 5$, to C2=0 →C2. New C1= $((2+1)/2, (3+2)/2) = (1.5, 2.5)$. New C2= $((5+6)/2, (4+5)/2) = (5.5, 4.5)$.

Q: [Hard] 1D points A=1, B=2, C=4, D=8, E=9. Complete-linkage agglomerative clustering. Where to cut for 3 clusters?
A: d(A,B)=1, d(D,E)=1 (tie; merge both). Step1: Merge A+B, h=1. Step2: Merge D+E, h=1. Clusters: {AB},{C},{DE}. d(AB,C)=max(3,2)=3. d(C,DE)=max(5,6)=6 (using C=4, D=8, E=9). d(AB,DE)=max(7,8,6,7)=8. Step3: min=3, merge AB+C, h=3. Step4: Merge ABC+DE, h=max(7,8,5,6)=8. For 3 clusters: cut between h=1 and h=3 → **{AB}, {C}, {DE}**.

13. 1

Search Algorithms

A*: $f(n) = g(n) + h(n)$. Admissible: $h(n) \leq h^*(n)$. Consistent: $h(n) \leq c(n, a, n') + h(n')$.
DFS Space: $O(bm)$. BFS/IDS Time: $O(b^d)$. IDS Space: $O(bd)$. UCS optimal for weighted graphs.

Neural Networks & Backpropagation

$\sigma(z) = \frac{1}{1+e^{-z}}$. $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$.
$\delta_{\text{out}} = (a_{\text{out}} - y)\sigma'(z_{\text{out}})$. $\delta_h = (\sum_k \delta_k w_{hk})\sigma'(z_h)$. $w \leftarrow w - \eta\delta a_{\text{prev}}$.
Loss: $L = \frac{1}{2}(y - \hat{y})^2$. Update: $w \leftarrow w - \eta\frac{\partial L}{\partial w}$.

Performance Metrics

Accuracy= $\frac{TP+TN}{N}$. Precision= $\frac{TP}{TP+FP}$. Recall= $\frac{TP}{TP+FN}$. $F_1 = \frac{2PR}{P+R}$.
Macro F1: average per-class F1. Micro F1: pool all TP/FP/FN first.

CNN

Output size: $\lfloor (N - F + 2P)/S \rfloor + 1$. Params: $(F^2 \cdot C_{\text{in}} + 1) \times$ filters.

Linear Regression

$w_1 = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}$. $w_0 = \bar{y} - w_1\bar{x}$.

Genetic Algorithm

Roulette: $P(i) = \frac{f_i}{\sum f_j}$. Rank: $P(r) = \frac{r}{N(N+1)/2}$. Simulated Annealing: $P(\text{accept worse}) = e^{\Delta E/T}$.